TECHNICAL DESIGN DOCUMENT

NESTORA

Smart Township Management System

Prepared By: Saran V

Resident app, help desk, gate and committee tools for a gated township in Pune, with MySQL persistence and optional Groq-assisted complaint triage.

GitHub repository	https://github.com/Sarancode6369/NESTORA_.git
Live application	https://nestora-new.vercel.app/
Version	3.0, final submission revision
Date	8 November 2026
Source reviewed	Repository commit 804c9c8 (6 November 2026)
Automated tests	122 run: 120 passed, 1 failed (mysql2 not installed in the test sandbox), 1 skipped (live Groq). SQLite, not MySQL.
Not verified	Live Groq API; production (Aiven) database; Vercel deployment's back end, sign-in and AI.

Contents

1. Project Overview 3

2. System Architecture and Technology Stack 5

3. Database Design and Data Model 8

4. Authentication and Role-Based Access 10

5. Resident Dashboard and User Interface 12

6. Complaint Management and Help Desk 13

7. Visitor Management and Gate Security 16

8. Maintenance Payments and Financial Management 18

9. Amenity Booking and Other Modules 20

10. AI Functionality and Automation 21

11. Security, Privacy and UX Decisions 25

12. Testing and Quality Assurance 26

13. Assumptions, Limitations and Future Improvements 30

14. Mobile Responsive Design 34

1. Project Overview

NESTORA replaces four things a Pune township of 2,400 apartments in 28 towers, run by 180 staff, currently does by hand: Excel maintenance bills, manual matching of bank payments, WhatsApp groups for complaints and a paper register at the gate. Monthly maintenance is the flat area multiplied by ₹6.50 per sq ft, so a 980 sq ft flat is billed ₹6,370 a month.

Item	Value
GitHub repository	https://github.com/Sarancode6369/NESTORA_.git
Live application	https://nestora-new.vercel.app/
Source reviewed for this revision	Commit *804c9c8* on 6 November 2026 (“Prepare NESTORA MySQL configuration and deployment”), read from a fresh clone of the repository above
Document date	7 November 2026

This revision of the document was written after reading the source, not only the previous draft. The earlier 19-page version described a build that had no resident assistant and no duplicate-complaint detection, and it carried 99 tests. The repository has since gained both features, a Vercel entry point and three more test files, so several numbers and sections have been corrected. Section 12 lists exactly what was run and what was not.

1.1 How to read the status words

Word	Meaning in this document
Implemented	The code is in the repository at the commit above. It says nothing about whether it was tested.
Tested	An automated test in *test/* exercises it. These tests run on in-memory SQLite and, for Groq, on an injected *fetch*; see section 12.
Observed	Visible in a screenshot or a manual run. A screenshot shows the interface only. It does not prove the database, the API or the AI provider behind it.
Not verified	Nobody has run it and recorded the result. Production MySQL, the live Groq API and the Vercel deployment’s back end are in this group.

1.2 Objectives

Goal	How the code meets it
Relational persistence	26 InnoDB tables with foreign keys, 25 index statements, 21 CHECK constraints and UNIQUE keys (section 3).
Money safety	One transaction per state-changing request. Payment reference, receipt number, bank transaction id, UTR and (flat, period) are UNIQUE in the schema.
AI that can be checked	Model output is validated field by field, retried once, and replaced by a keyword classifier when it fails. A person reviews uncertain results (section 10).

Goal	How the code meets it
Emergency safety	Regular expressions detect fire, gas leak, lift entrapment and electric shock without asking the model. The model can raise a priority but never lower a rule-triggered one.
Role separation	Four roles (resident, help desk, committee, guard). Every route lists the roles allowed to call it.

1.3 Modules covered

OTP sign-in and roles (4), the landing page and resident home (5), complaints, the help desk and duplicate suggestions (6), visitor management at the gate (7), bills, test-mode payments, bank reconciliation and the committee dashboard (8), amenity bookings, notices and notifications (9), the AI features (10), security (11), tests (12), limits (13) and the phone layout (14). The app is a progressive web app in English, Hindi and Marathi.

2. System Architecture and Technology Stack

The browser talks JSON over HTTPS to one Node.js HTTP handler. That handler checks the token and the role, validates the input and calls plain service modules. Services call repositories, which are the only place SQL is written. The database adapter hides whether the connection is MySQL or SQLite. The AI layer sits beside the services and is always optional: every path that calls Groq has a rule-based answer to fall back on (Figure 1).

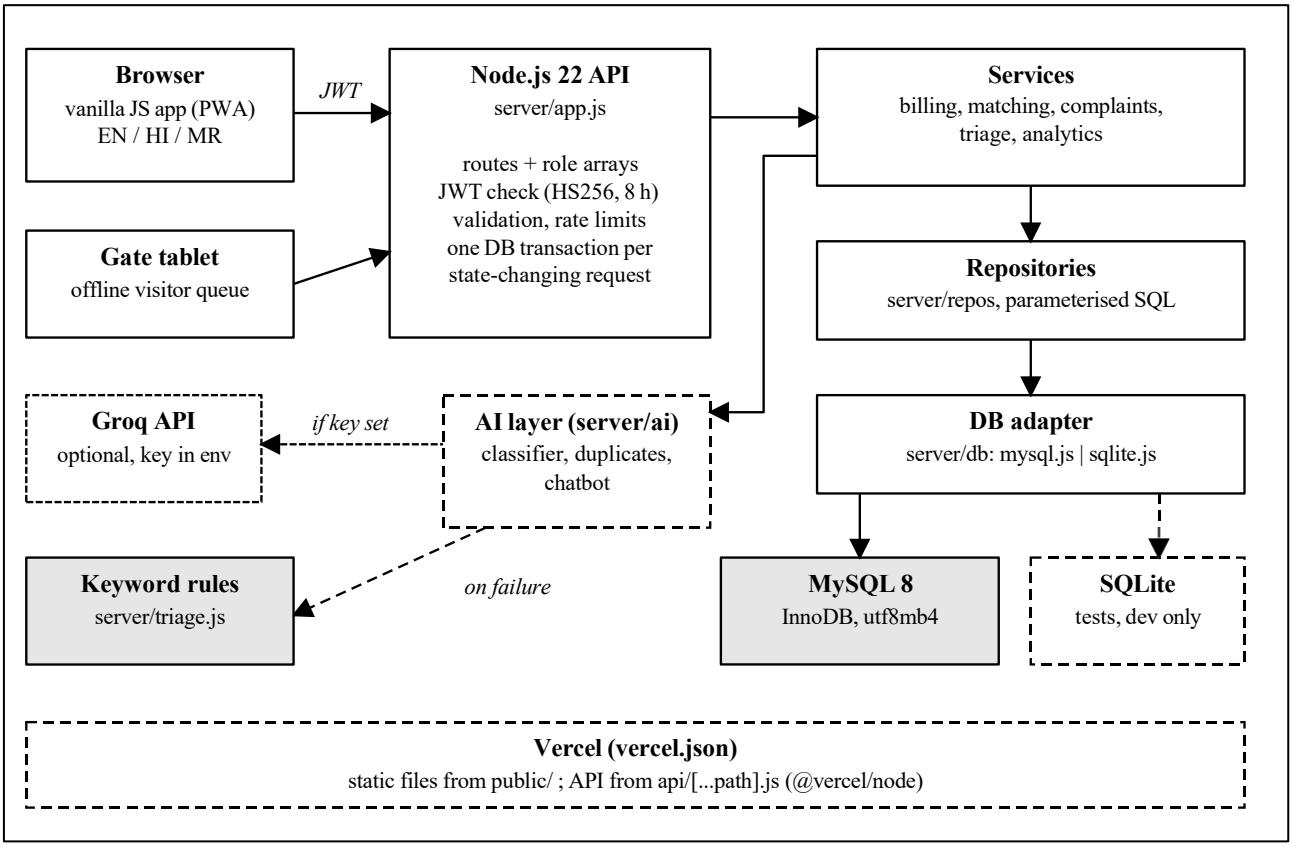


Figure 1: NESTORA architecture: browser, Node API, services, repositories, database adapter, optional Groq and the rule-based fallback. Dashed boxes are optional or alternative paths.

2.1 Decisions

Decision	Choice	Reason
Runtime	Node.js 22, ES modules, no web framework	The built-in *fetch* and *node:test* are enough. *mysql2* is the only dependency in *package.json*, and *engines* asks for Node 22 or newer.
Database	MySQL 8, InnoDB, utf8mb4	Needs transactions, foreign keys and CHECK constraints (enforced from 8.0.16).
Data access	Repository layer, parameterised SQL, no ORM	All SQL lives in *server/repos/*. Services never build SQL strings.
Engine switch	*server/db/index.js* picks MySQL unless *engine: 'sqlite'* is passed	Tests and *npm run dev:sqlite* ask for SQLite explicitly. *npm start* and *npm run db:** use MySQL. The same *schema.sql* is applied to both.

Decision	Choice	Reason
Identifiers	Readable ids (CMP-0013, PS-00001) from a *counters* table	Keeps ids and receipts human-readable; the counter row is updated inside the same transaction as the record.
Money and time	DECIMAL(12,2), DATE, DATETIME(3)	The MySQL driver is configured with *decimalNumbers* and *dateStrings* so values are not converted to floats or local time.
Auth	HS256 JWT written in *server/app.js*, demo OTP	The brief asks for a mock OTP. Roles are enforced per route.
AI provider	Groq chat completions, default model *openai/gpt-oss-120b*	Configurable through *GROQ_MODEL*. A comment in *groq.js* notes that the earlier default was retired by Groq.

2.2 Front end

The front end is static files in *public/* with no build step: *app.js* (hash router and shell), *views-resident.js*, *views-staff.js*, *views-gate.js*, *ui.js* (an escaping template helper and shared widgets), *i18n.js* (English, Hindi, Marathi), *charts.js*, and for the landing page *journey.js* (scroll-driven image sequence) and *township3d.js* (interactive 3D township). A service worker caches the application shell only. API calls are never cached.

2.3 Deployment layout

vercel.json declares two builds. *public/*** is served as static files with *@vercel/static*, and *api/[...path].js* runs the API with *@vercel/node*. Routes send */api/** to the function and everything else to *public/*. On the first request an instance opens the database, applies the schema, loads demo data if the database is empty (unless *SEED_DEMO_DATA=false*) and builds the app once. The function refuses to start without *JWT_SECRET* and turns the demo-reset route off. A second file, *api/index.js*, exists with a similar handler, but *vercel.json* routes only to *[...path].js*. Whether the live site at the address on the cover is currently wired to a working production database has not been verified (section 12.5).

2.4 Repository layout

Path	Contents
server/app.js	Routes, JWT, rate limiters, validation, response headers
server/billing.js, matching.js, complaints.js, triage.js, analytics.js	Business rules: statements and late fees, bank CSV matching, complaint lifecycle, keyword triage and safety rules, committee numbers
server/ai/	*groq.js* client, *classifier.js*, *validate.js*, *duplicates.js*, *chatbot.js*, *index.js* (builds the classifier and status from environment)
server/db/	*config.js* (environment to connection settings), *mysql.js*, *sqlite.js*, *bootstrap.js* (init, seed, reset), *import.js* (legacy JSON import)
server/repos/	core, billing, complaints, community repositories

Path	Contents
database/	*schema.sql*, *seed.sql*, *fixtures/*
scripts/	*db.js* (init, seed, reset, import, export), *dev-sqlite.js*
api/, *vercel.json*	Vercel entry points and routing
test/	9 test files plus *helpers.js*

3. Database Design and Data Model

database/schema.sql creates 26 tables. It uses *CREATE TABLE IF NOT EXISTS*, so running it twice is harmless, and each run is recorded in *schema_migrations*. Sample data comes from *npm run db:seed*, which refuses to run on a database that already has users. *import-json* loads an older JSON snapshot, mapping the old Housekeeping category to Cleaning and the old Critical priority to Emergency.

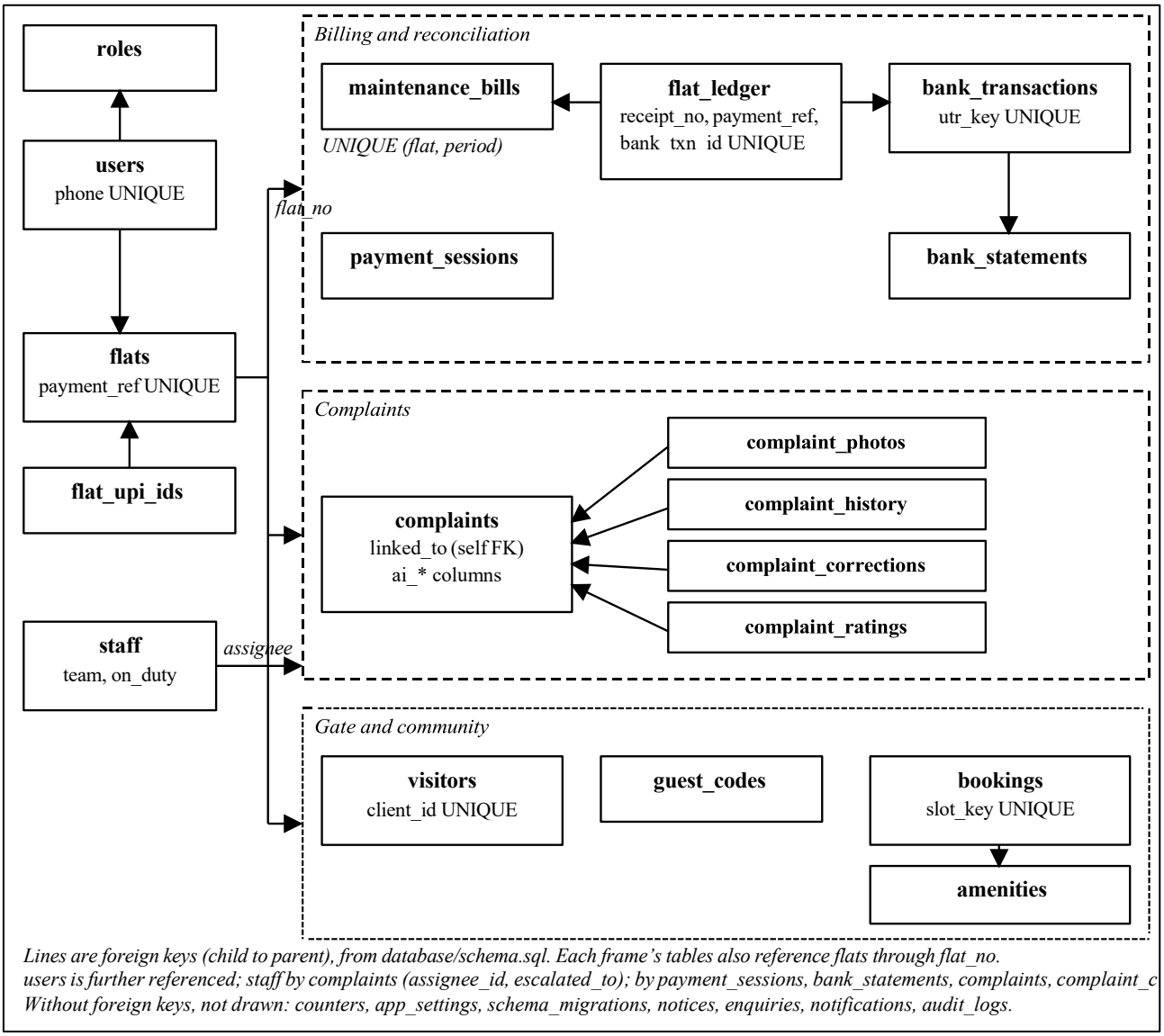


Figure 2: Entity relationships taken from the foreign keys in database/schema.sql (26 tables; 19 shown with relationships).

3.1 Tables by purpose

Group	Tables	Notes
Reference	*roles*, *app_settings*, *counters*, *schema_migrations*	The rate, due days and late-fee policy live in *app_settings* and are edited through *PUT /api/config*, not hard-coded.

Group	Tables	Notes
People and property	*flats*, *flat_upi_ids*, *users*, *staff*	A flat has a unique payment reference. A phone number belongs to one user. Users point to a role and optionally a flat.
Billing	*maintenance_bills*, *flat_ledger*, *payment_sessions*	One bill per flat and period. The ledger holds bills, late fees and payments as rows.
Reconciliation	*bank_statements*, *bank_transactions*	Each imported row has a status (matched, partial, unmatched, needs_review, duplicate), a reason and a confidence.
Complaints	*complaints*, *complaint_photos*, *complaint_history*, *complaint_corrections*, *complaint_ratings*	Classification method, model, confidence, explanation and review flag are columns on the complaint. *linked_to* is a self-reference used for duplicates.
Community	*visitors*, *guest_codes*, *notices*, *amenities*, *bookings*, *enquiries*	Offline replays are made idempotent by the unique *client_id*. One booking per slot is enforced by the unique *slot_key*.
Governance	*notifications*, *audit_logs*	Append-only records of financial, reconciliation, correction, link and retention events.

3.2 Constraints that protect money

Constraint	What it stops
flat_ledger.payment_ref UNIQUE	Posting the same payment reference or UTR twice
flat_ledger.receipt_no UNIQUE; *flat_ledger.bank_txn_id* UNIQUE	Duplicate receipts; one bank transaction being credited to two ledger rows
bank_transactions.utr_key UNIQUE (set only on the canonical row)	Two live rows for one UTR. A re-imported row is stored with status duplicate and a NULL key.
maintenance_bills UNIQUE (flat_no, bill_period)	Billing a flat twice for the same month
bookings.slot_key UNIQUE	Two live bookings for one slot, even under a race
21 CHECK constraints (enums for status, category, priority, method; amounts not negative; confidence from 0 to 1)	Bad values reaching storage even if application code has a bug

Indexes cover the common reads: ledger by (flat, date) and (kind, date), complaints by status, (category, priority), (assignee, status), creation time and review flag, bank transactions by status, fingerprint and flat, and the audit log by time and action. These constraints are written in the schema and were applied by the local MySQL initialisation described in section 12.3. The automated tests that probe them (duplicate UTR, concurrent posting, atomic rollback) ran on SQLite, so their behaviour on MySQL under concurrency is not covered by a recorded test run.

4. Authentication and Role-Based Access

4.1 Sign-in flow

A user enters the mobile number linked to a flat or a staff record (Figure 3). *POST /api/auth/request-otp* answers with the demo OTP, which is shown on the next screen because no SMS is sent (Figure 4). *POST /api/auth/verify-otp* compares the code with the configured demo value, allows five wrong attempts per request, and expires a request after ten minutes. On success the server returns an HS256 token with the user's role, valid for eight hours. Both screens carry a demonstration-mode notice.

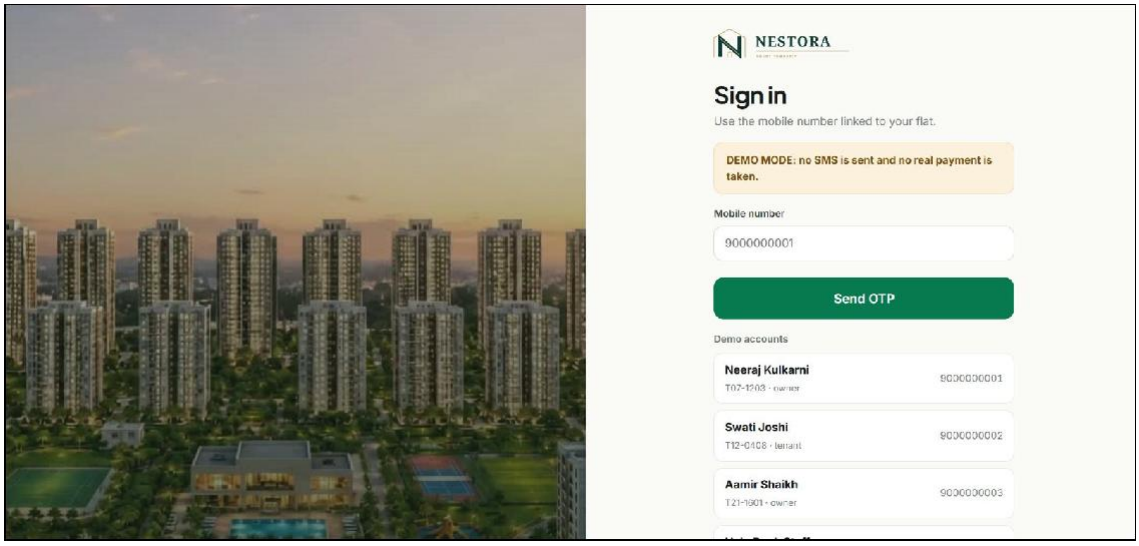


Figure 3: NESTORA sign-in screen with the demo-mode notice and the list of demo accounts.

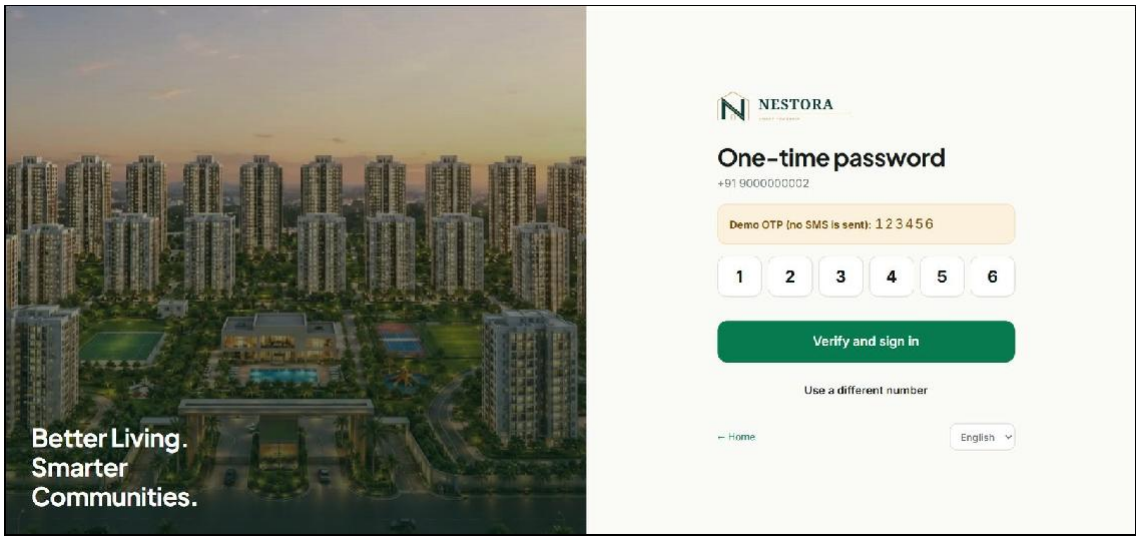


Figure 4: OTP verification screen. The demo OTP is displayed because no SMS is sent.

4.2 Roles

Role	What it can reach (from the route definitions in *server/app.js*)
Resident	Own flat dashboard, ledger, bills and receipts; test-mode payments; raising, rating and reopening own complaints; duplicate suggestions and the AI complaint analysis; the resident assistant (*/api/chat* is resident-only); visitor approval and guest codes; amenity bookings. A resident asking for another flat's ledger or dashboard gets 403; another flat's complaint or visitor gets 404.
Help desk	The complaint queue and every complaint action (category, priority, confirm, status, assign, link, unlink); bills, late fees and settings; bank reconciliation; visitors and notices; analytics, audit and notifications; the demo reset route when not in production.
Committee	Analytics and the monthly CSV report, audit log, configuration and AI status (read only), notices.
Guard	Visitor registration, entry and exit, guest-code redemption.

A route × role test calls each route with each role and expects 403 for the wrong role and 401 for a missing or tampered token. Token tampering and expiry have their own tests (*security.test.js*, *api.test.js*).

5. Resident Dashboard and User Interface

The public landing page introduces the township and leads to sign-in (Figure 5). It has the navigation (Home, 360°, About, Services, Amenities, Notices, Contact), a language selector, a Sign In button and the headline figures: 2,400 residences, 28 towers, 180 staff.

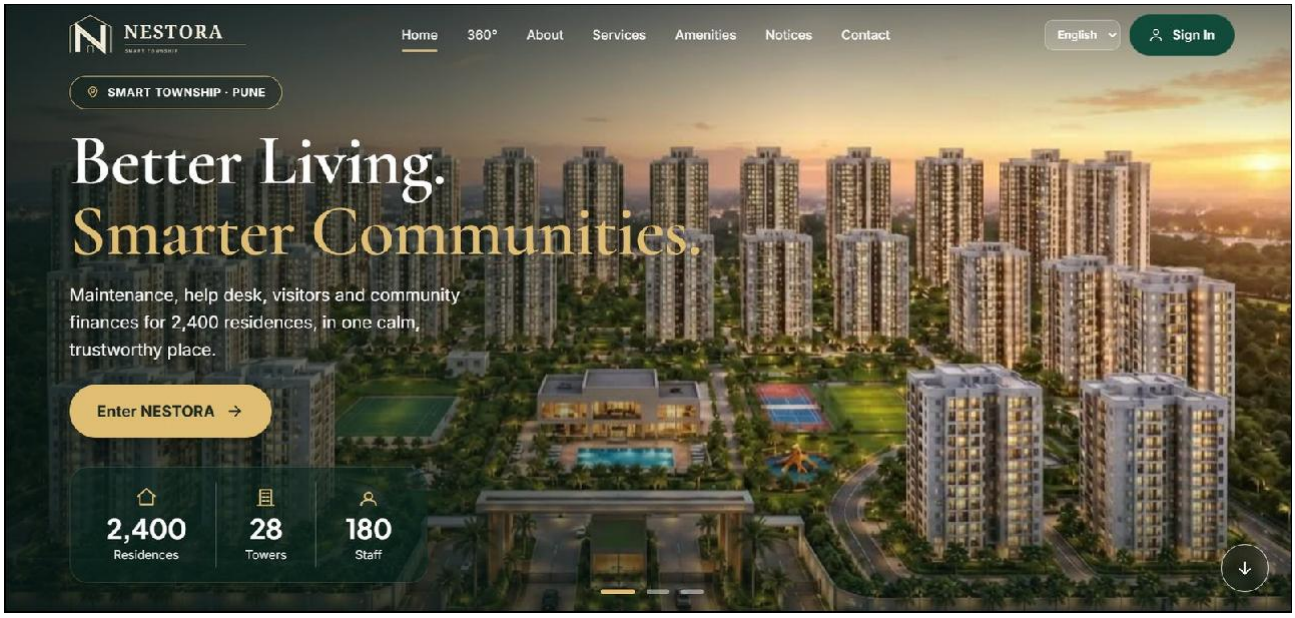


Figure 5: NESTORA home page with navigation, language selector and the headline figures.

After sign-in a resident sees six destinations in the current build: Home, Pay, Complaints, Visitors, Book and AI Assistant. The home screen (Figure 6) shows what needs attention first: maintenance due with an overdue badge, open complaints and visitors today, followed by quick actions and community notices. In the capture, Swati Joshi's flat T12-0408 (980 sq ft) has one unpaid bill of ₹6,370, 49 days overdue, and one open complaint, "Water leak · Assigned". All names and records in the screenshots are demonstration data.

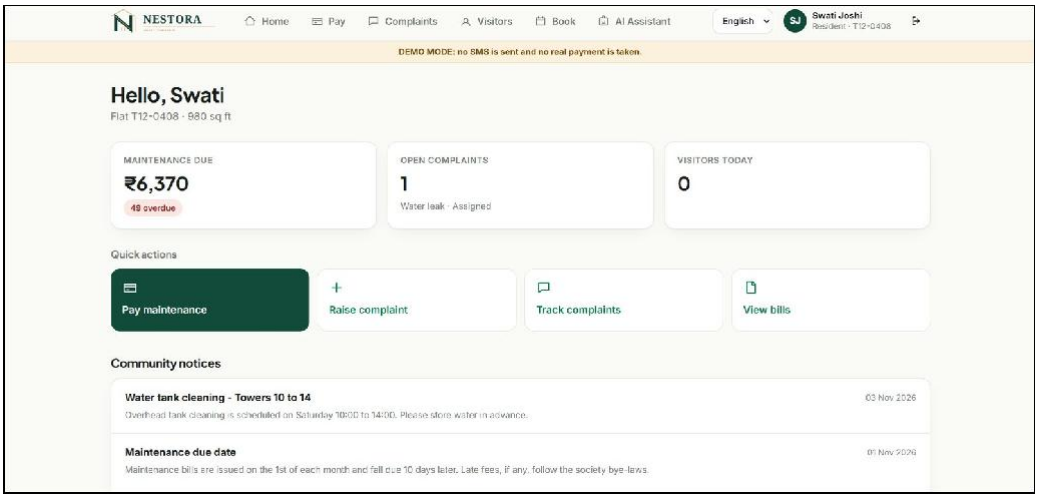


Figure 6: Resident home for flat T12-0408: maintenance due, open complaints, visitors today, quick actions and notices.

The guiding rule for the interface is that the most useful number is on the first screen, and that anything an automated step decides can be seen and changed by a person (section 11).

6. Complaint Management and Help Desk

6.1 Raising a complaint

A resident writes a title and description, optionally picks a category and attaches up to three photos (Figure 7). The form has an Analyze with AI action beside the description field. It calls POST /api/complaints/analyze, which returns a suggested category, priority, short summary, confidence and, when a safety rule fires, a safety warning (Figure 8). The resident can change the suggestion before submitting. The category is optional because the classifier works it out from the text when it is left blank. Analysis calls are limited to 60 a minute per user and complaint creation to 10 a minute.

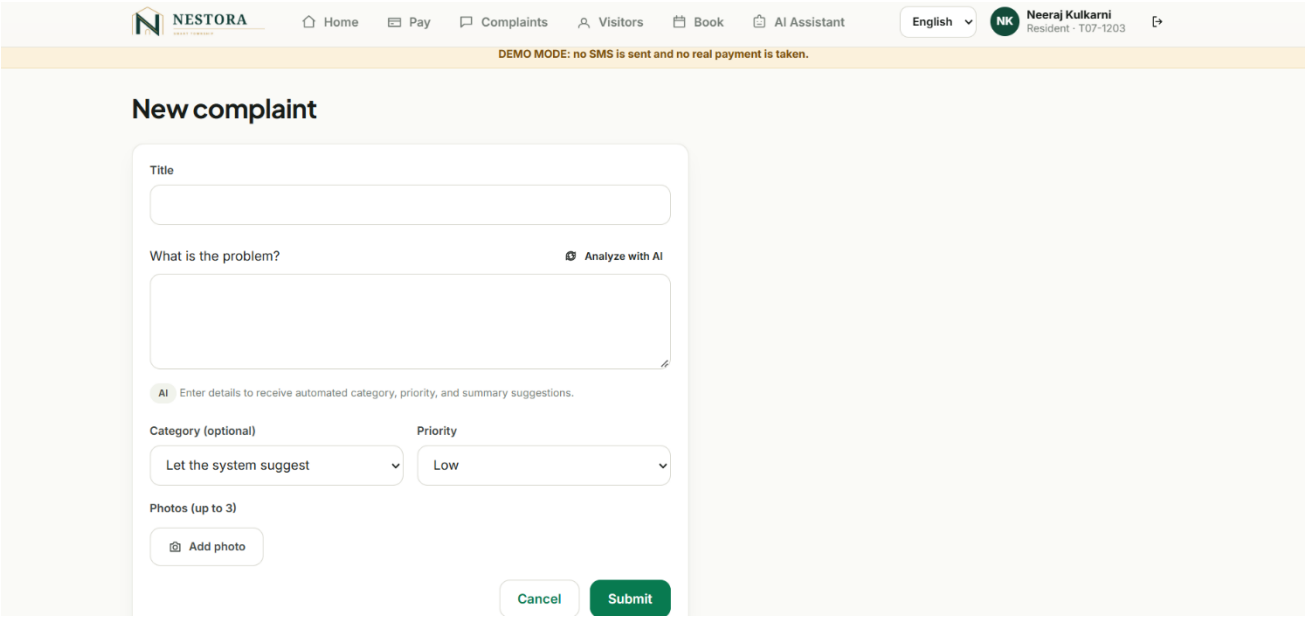


Figure 7: New complaint form: title, description with the Analyze with AI action, optional category, priority and photo upload.

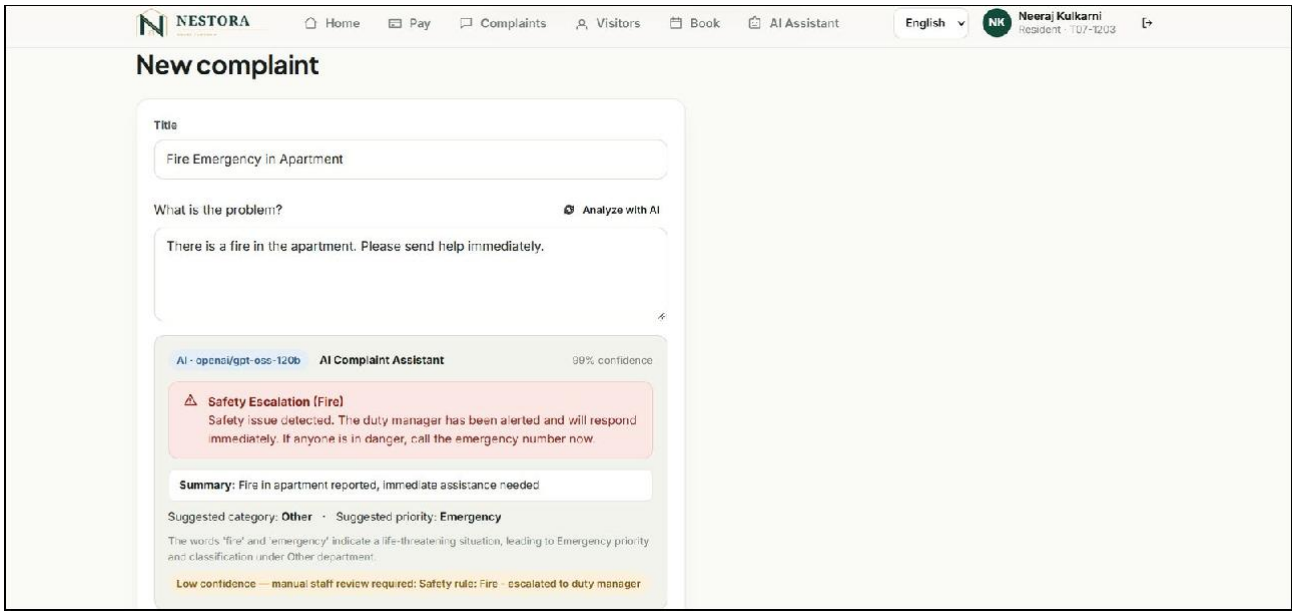


Figure 8: New complaint form after Analyze with AI on a fire report: safety escalation notice, summary, suggested category and priority, and a manual-review notice. The badge reads “AI · openai/gpt-oss-120b”.

Figure 8 is a screenshot, so it shows what the interface displayed at that moment. It does not by itself prove that a live Groq request was made from the environment that was captured, and no test run recorded in this document repeats it (section 12.4). The emergency treatment in the capture comes from the rule engine described in section 10.2, which does not depend on the model.

6.2 Lifecycle

A complaint is classified, stored, and offered to the duty manager if it is an emergency. Staff can correct the category or priority, confirm an uncertain classification, assign it, move it through statuses, link it to a duplicate, or leave it for a person to resolve. The resident can rate a resolved complaint or reopen it. Nothing closes a complaint automatically.

6.3 Help desk screens

Help-desk staff see tiles for Open, Needs review, Response overdue and Emergency, filters for status, category and urgency, a “Needs review only” toggle and a table with flat, category, urgency, status, assignee and response time (Figure 9). An escalated complaint is also pinned in a banner above the tiles.

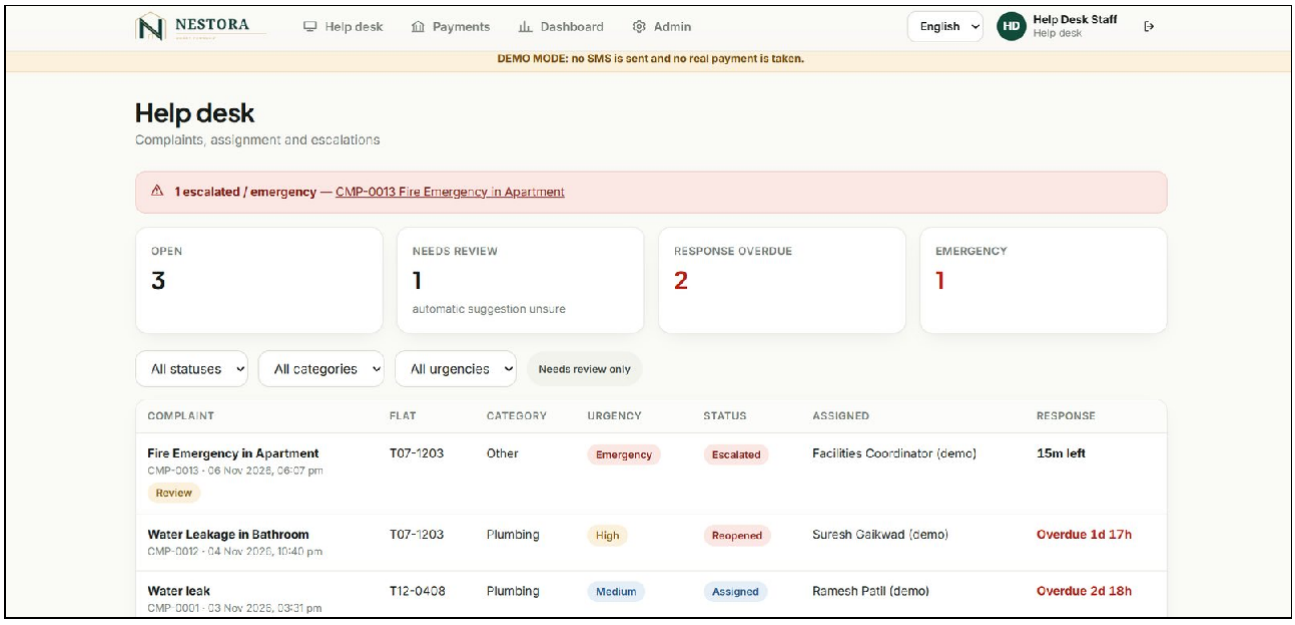


Figure 9: Help desk queue with an escalated fire complaint (CMP-0013, “Review” flag), summary tiles and filters.

Opening a complaint shows the classification card (Figure 10): which method produced it (AI, keyword fallback or corrected by staff), the model name, category, priority and confidence, the explanation, any review reasons, a one-tap *Confirm classification is correct* button, category and priority controls and the assignment fields. When a safety rule raised the priority, the drawer says that it cannot be lowered. Residents never receive the reasons, explanation, model name or review reasons; the API removes them from resident responses.

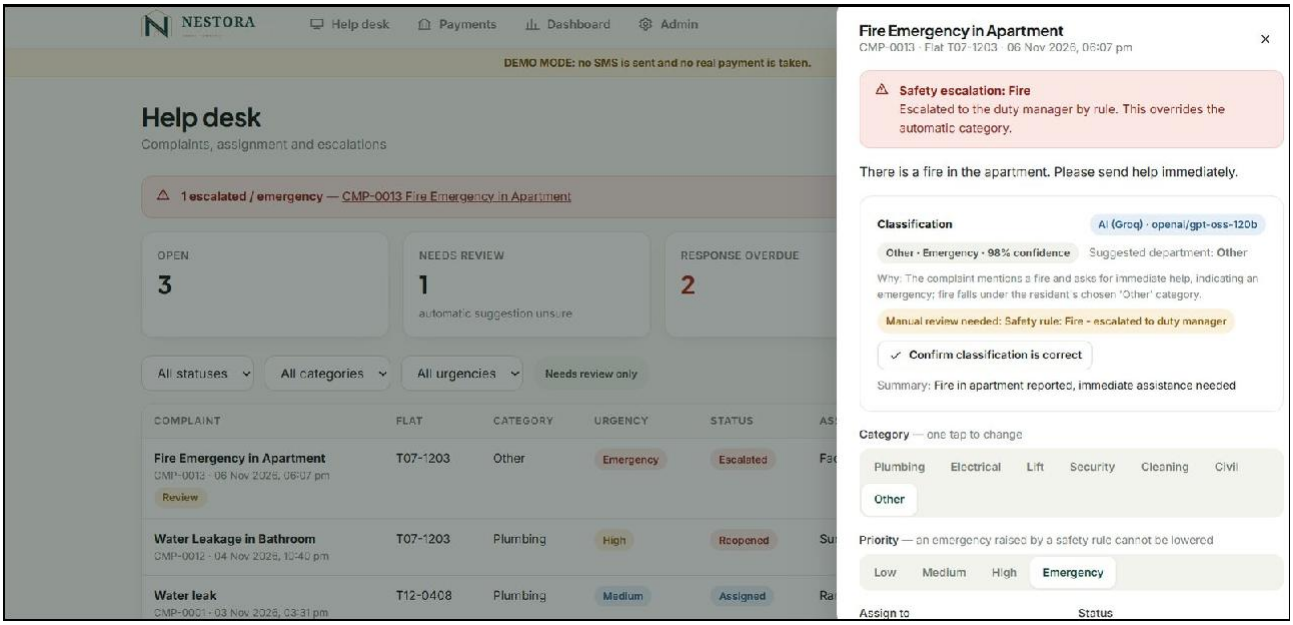


Figure 10: Help desk complaint drawer for CMP-0013: safety escalation, classification card with “AI (Groq) · openai/gpt-oss-120b”, confirm button and priority control.

6.4 Duplicate complaints

Before a complaint is submitted, *POST /api/complaints/duplicates/suggest* compares it with open complaints. *server/ai/duplicates.js* first scores candidates without any model: same flat, floor, tower or shared area, same category, and word overlap. The best few (up to five) are then compared by Groq for meaning, and the final score blends the two (70% model, 30% rule score). If Groq is unavailable the rule score is used alone. One wiring detail matters on Vercel: *server/index.js* passes the Groq client to *createApp* for this route, but *api/[...path].js* does not, and the duplicate route uses only the client it is given. On the Vercel entry point the duplicate suggestions therefore appear to use the rule score even when a key is set. This was read from the source, not observed. Scores below 0.6 are flagged for review. Residents get the matching complaints without other residents’ flat or tower; staff see them.

A duplicate is only ever *linked*. Help desk staff confirm a link with *POST /api/complaints/:id/link* and can undo it with */unlink*. The tests check that linking needs staff action, does not close or hide anything, rejects circular links and links to resolved complaints, keeps emergency priority, and writes audit entries for both link and unlink.

7. Visitor Management and Gate Security

The gate screen is a separate dark interface for guards (Figure 11). It has *New visitor* and *Guest code* actions, a “Waiting for resident” list, an “Inside now” list and an Online indicator.

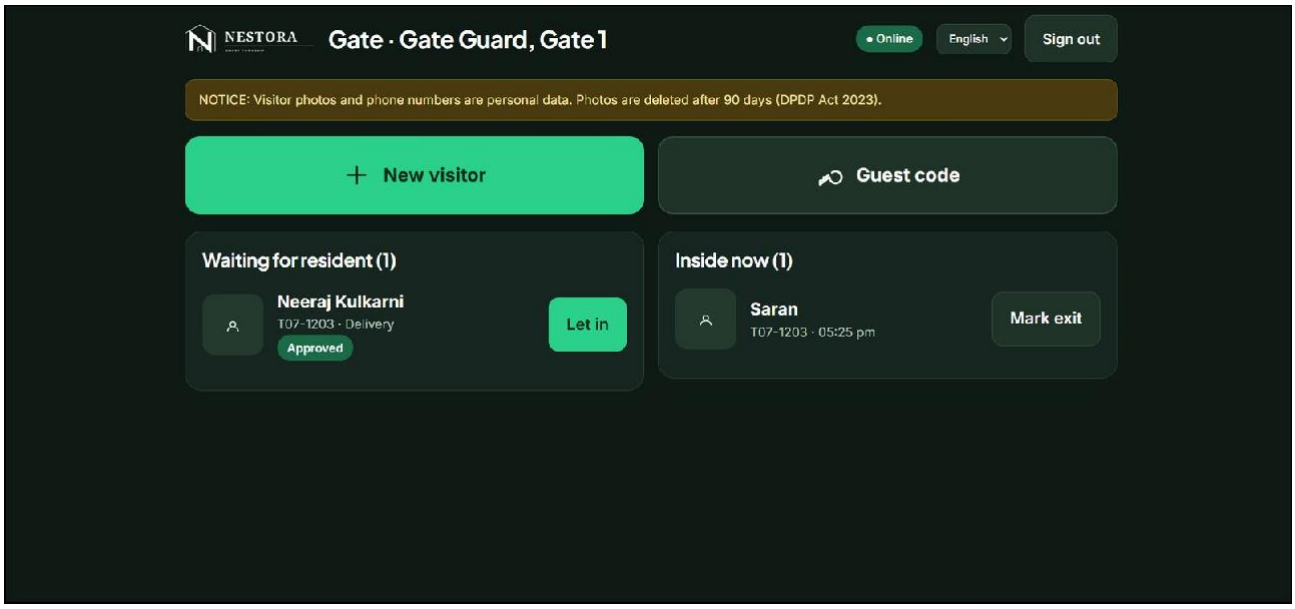


Figure 11: Gate guard screen: waiting for resident (approved delivery, Let in) and inside now (Mark exit).

Step	What happens
Register	The guard registers a visitor against a flat and a purpose.
Decide	The resident approves or denies (*POST /api/visitors/:id/decision*).
Enter and exit	The guard records entry and exit. Only approved visitors can be let in.
Guest code	A resident creates a six-digit pre-approved code for a named guest and date (Figure 12). The guard redeems it at the gate; a used code shows as Used.
Offline	If the tablet is offline, new visitors are queued in the browser’s local storage with a client id and replayed later. *visitors.client_id* is UNIQUE, so a replay cannot create a second row. Server errors keep an item in the queue; a rejected (400) item is dropped so the queue cannot jam.

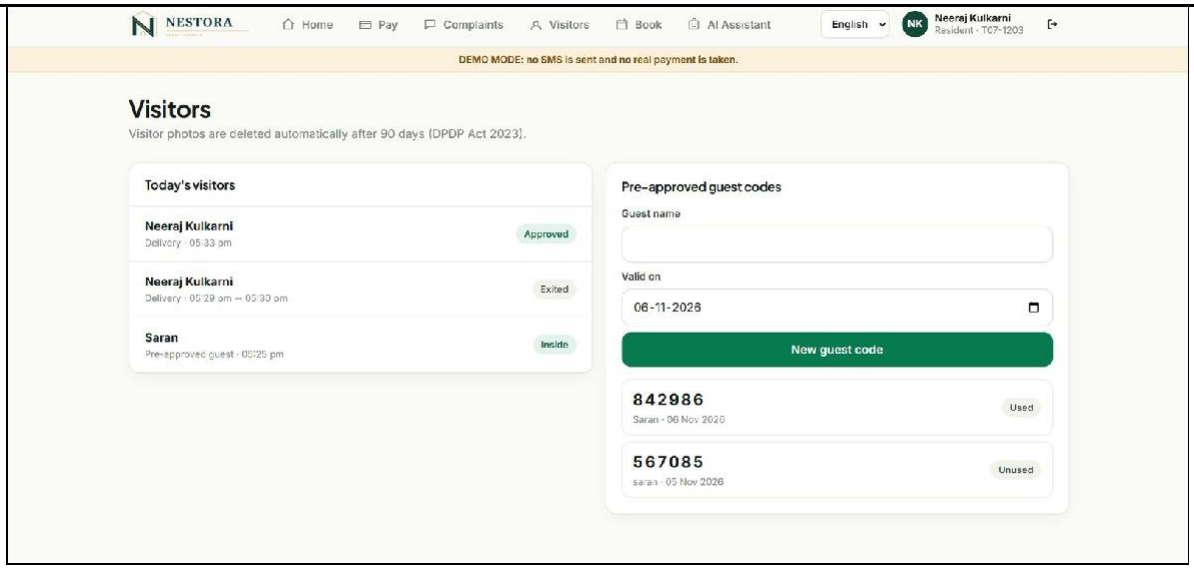


Figure 12: Resident Visitors page: today's visitors with their status and pre-approved guest codes.

The gate screen shows a privacy notice: photos and phone numbers are personal data and photos are deleted after 90 days (DPDP Act 2023). An hourly job started with the server performs the deletion and records it in the audit log. On Vercel, a function that only runs when a request arrives may not run an hourly timer reliably; this has not been checked. The offline queue was read in the source but a browser test of an offline replay is not part of the test suite.

8. Maintenance Payments and Financial Management

8.1 Resident payment screen

The Pay page has three tabs (Bills, Ledger, Receipts) and a *Pay now* button showing the outstanding amount (Figure 13). Each bill row shows the month, the area × rate basis, amount, paid, balance and a status badge. For the flat in the capture each month is 980 × 6.50 = ₹6,370; September 2026 has ₹0 paid and an overdue badge, and April to August are paid.

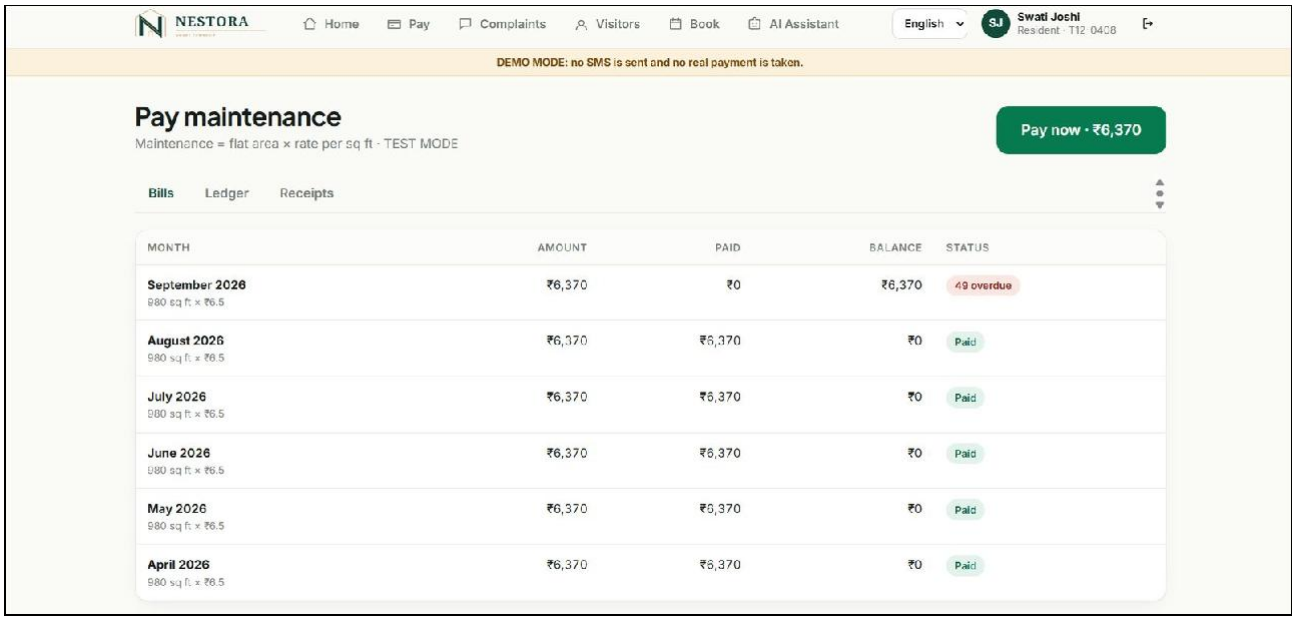


Figure 13: Pay maintenance page in test mode: bills with paid, balance and status.

8.2 Payment flow

Payments are test-mode only; the screen says so and no card or bank details are collected. *POST /api/payments/checkout* creates a session for an amount not above the outstanding balance. *POST /api/payments/:id/complete* flips the session from *created* to *paid* with a single conditional UPDATE, so of two simultaneous completions only one succeeds, and then posts the ledger row and receipt in the same transaction. Partial payments are allowed and settle the oldest charge first. Late fees follow a configurable mode: none, flat, percent per month or percent per annum. None is applied by default.

8.3 Bank statement reconciliation

Help-desk staff upload a bank CSV. Rows are parsed and validated, checked against earlier rows by fingerprint and UTR, and matched to flats by payment reference, UPI id and amount. The outcome is matched, partial, needs_review, unmatched or duplicate. An exact amount alone never confirms a match, and a UPI id with an amount but no reference goes to review. Uploading never posts anything. Posting happens only when staff approve a row, or press auto-post, which takes only rows with status *matched* and confidence of at least 0.9. Each approval, rejection and auto-post is written to the audit log. A sample file is in *samples/bank-statement-sample.csv*.

8.4 Committee dashboard

Totals, ageing, resolution time and reopen rate are computed from ledger and complaint rows (*server/analytics.js*), and a monthly CSV can be downloaded. In the capture (Figure 14) the page states that its figures come from the demonstration data set of five flats. Billed ₹2,26,980, collected ₹2,01,625 (88.83%) and outstanding ₹25,355 across 4 of 5 flats are therefore sample numbers, not township actuals.

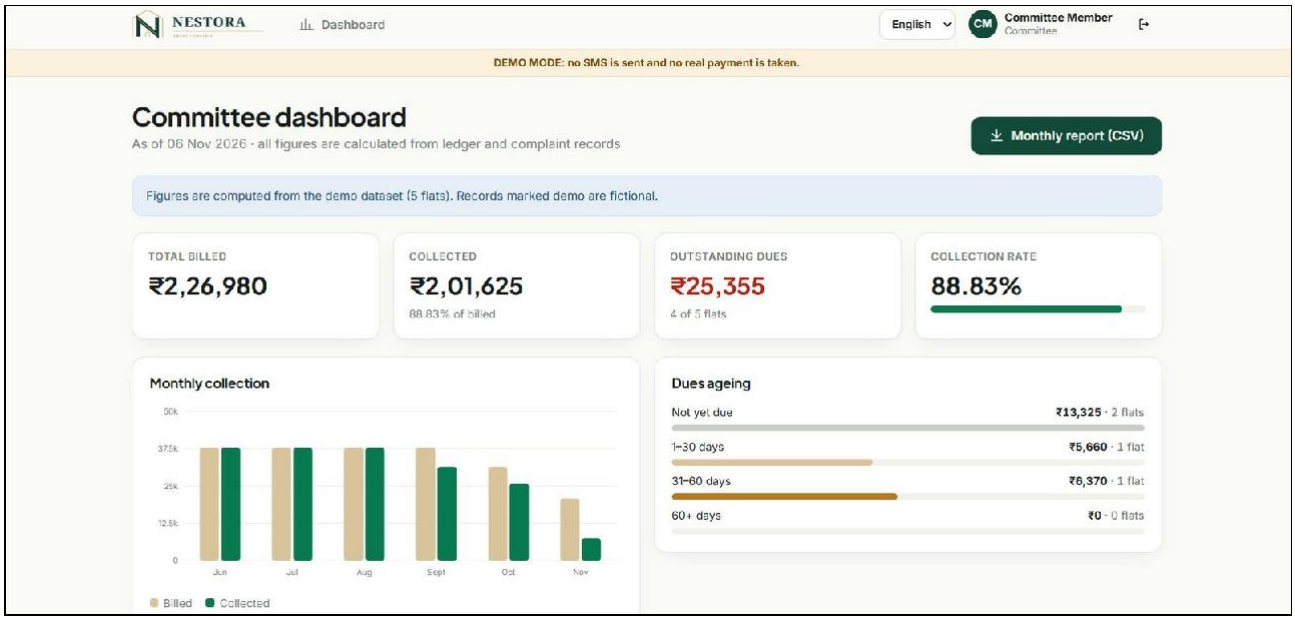


Figure 14: Committee dashboard: billed, collected, outstanding, collection rate, monthly collection and dues ageing (demonstration data).

9. Amenity Booking and Other Modules

Amenity booking. The Book page lists the clubhouse hall, badminton court and tennis court with fee, rules and hourly or two-hour slots for a chosen date; taken slots are greyed out (Figure 15). The unique *slot_key* blocks a double booking, one slot per flat per day applies to the courts, and cancellation closes 24 hours before the slot.

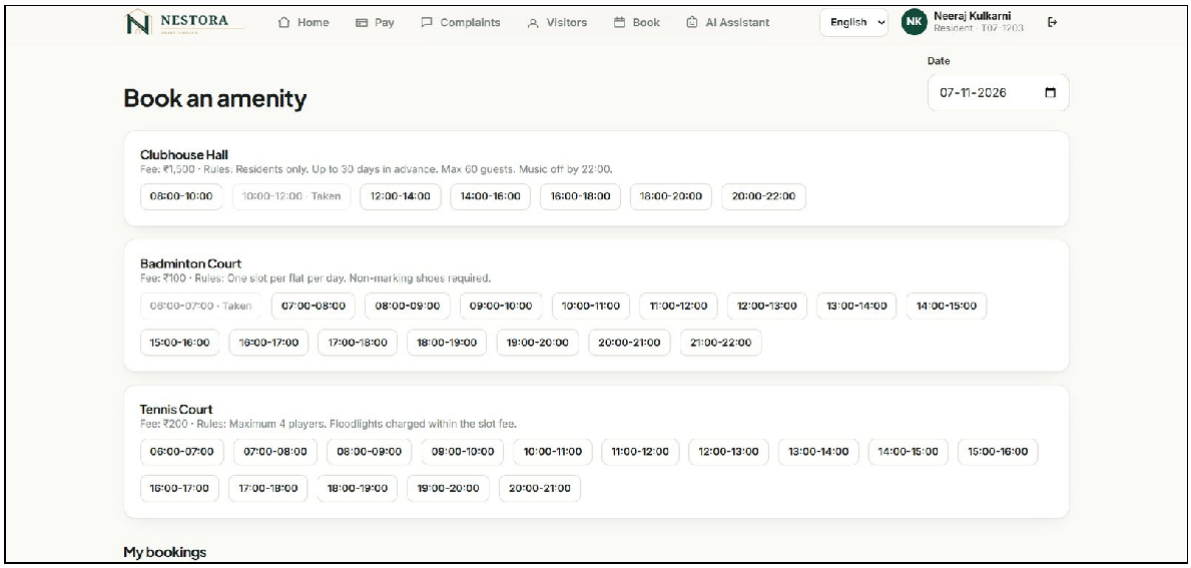


Figure 15: Book an amenity: slots for the clubhouse, badminton and tennis courts with taken slots shown.

Notifications. Stored in *notifications* and read by the help desk, for example when a complaint is escalated as an emergency. **Notices and enquiries.** *notices* feeds the resident home and landing page; *enquiries* stores the public contact form, which has its own rate limit. **Audit.** *audit_logs* keeps financial, reconciliation, correction, link and retention events and is readable by the help desk and committee.

10. AI Functionality and Automation

NESTORA has three AI-assisted features, all in *server/ai/* and all optional. They share one Groq client. Without *GROQ_API_KEY* the app runs on rules alone and makes no network call to Groq.

Feature	Route	Model used for	Without the model
Complaint classification (on submit) and the Analyze with AI preview	*POST /api/complaints*; */api/complaints/analyze*, */api/ai/analyze*	Category, priority, summary, suggested department and staff, confidence, explanation	Keyword classifier in *server/triage.js*; method stored as *keyword_fallback*
Duplicate suggestions	*POST /api/complaints/duplicates/suggest*	Comparing a new complaint with up to five prefiltered candidates	The rule score alone
Resident assistant	*POST /api/chat* (residents only)	Writing a plain-text answer from the resident's own data	A fixed apology that points to the help desk; no rule-based answers

10.1 Complaint classification

Figure 16 shows the classification path. Steps 1 and 5 are plain rules; steps 2 to 4 involve the model call and its checks.

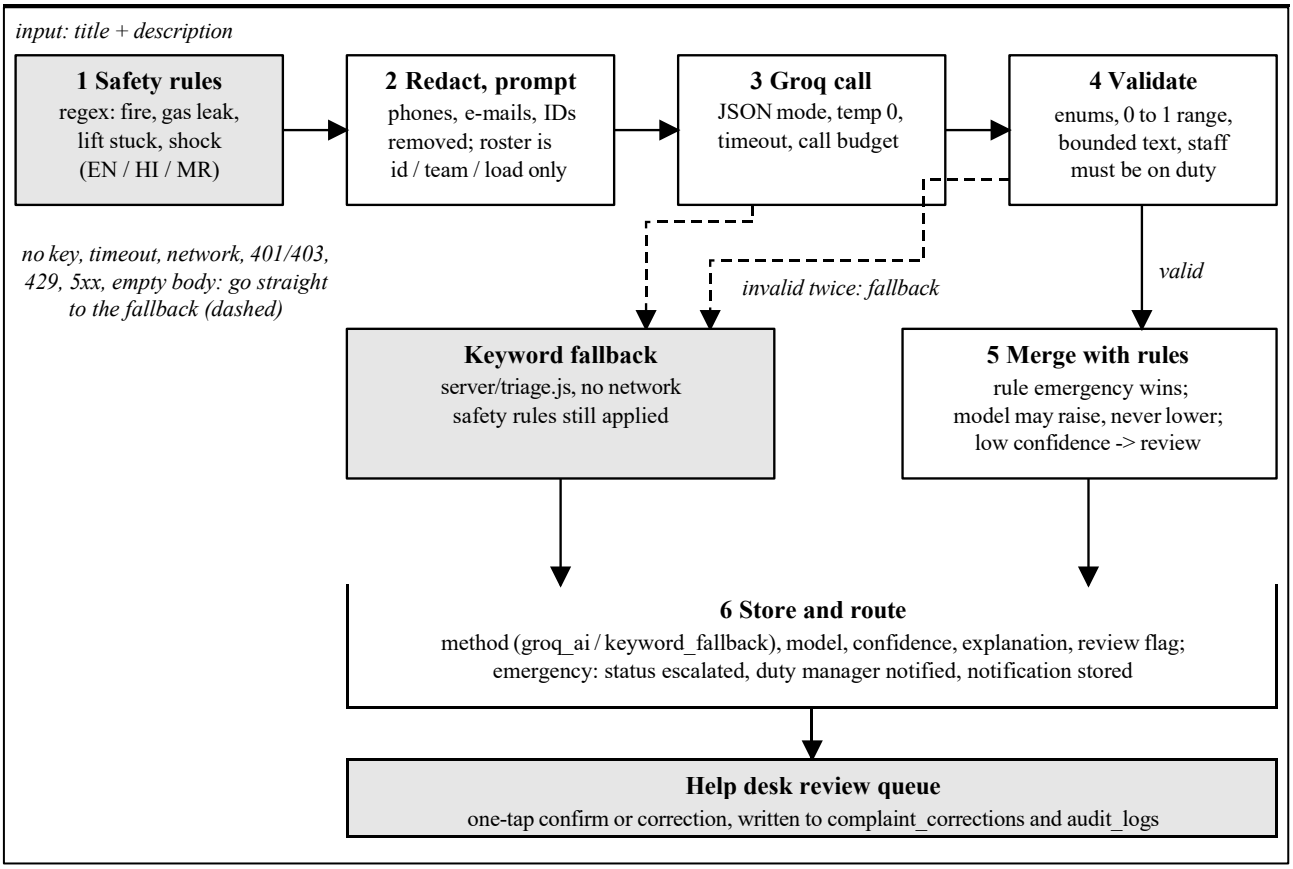


Figure 16: Complaint classification workflow in server/ai/classifier.js: rule-based safety check, redaction, one Groq call, validation, one retry, keyword fallback and merge.

Step	Detail (from *classifier.js*, *groq.js*, *validate.js*)
Input	Title (120 characters) and description (1,500). E-mail addresses, 10-digit mobile numbers and 12-digit ID numbers are replaced with placeholders. The prompt carries no resident name or flat. The staff roster is sent as id, team and open-complaint count only. The complaint text is wrapped in tags and the prompt says it is untrusted.
Request	Temperature 0, JSON mode, a timeout (8 s default, 0.5 to 30 s allowed), at most 1,200 completion tokens. For *openai/gpt-oss* models a low reasoning effort is set. The key goes only in the Authorization header.
Required JSON	category (Plumbing, Electrical, Lift, Security, Cleaning, Civil, Other), priority (Low, Medium, High, Emergency), summary, suggested_department, suggested_staff_id, confidence (0 to 1), explanation.
Validation	Enum membership, numeric range, text length (long text is trimmed). A staff suggestion survives only if that person is on duty in the chosen department. Anything else rejects the whole answer. A fenced or prose-wrapped JSON object is tolerated; garbage is not.
Retry	An invalid answer is retried once with the validation errors added to the prompt. If the model rejects JSON mode with HTTP 400, the call is repeated once without it. A second failure ends in the keyword fallback.
Stored	ai_category, ai_priority, ai_confidence, ai_explanation, classification_method (groq_ai, keyword_fallback or manual), classification_model, suggested_department.

Step	Detail (from *classifier.js*, *groq.js*, *validate.js*)
Review	Confidence below *AI_MIN_CONFIDENCE* (default 0.6), disagreement with a confident keyword result, or a conflicting resident choice sets needs_review with the reasons. These show in the help desk review queue.
Correction	Staff change category or priority in one action; the method becomes manual, a row goes to *complaint_corrections* and *audit_logs*, and the history is extended.

10.2 Emergency rule engine (no model involved)

Regular expressions in *server/triage.js* cover fire, gas leak, a lift stuck or trapped, and electric shock, in English, Hindi and Marathi. They run on the raw text before and independently of the model, and their result is applied after it:

- A rule hit forces priority Emergency, status *escalated*, assignment to the duty manager and a stored notification.
- The model's answer is merged one way. It may raise a complaint to Emergency (also escalated, but correctable). It cannot lower a rule-triggered emergency, whether it answers Low, fails or returns junk, and text inside the complaint cannot talk it out of escalating.
- Staff cannot lower a rule-triggered emergency either; a person resolves it. Complaints are never auto-closed.

10.3 Failure handling

Condition	Behaviour	Complaint
No *GROQ_API_KEY*	Keyword classifier, no network call	Created, keyword_fallback
Network error, timeout, 401/403, 429, other HTTP error, empty or non-JSON body	Keyword fallback; only the error kind is logged, never the key or the text	Created, keyword_fallback
Invalid JSON or unknown category	One retry, then fallback; bad data is never stored	Created, keyword_fallback
Local call budget used up (*AI_MAX_CALLS_PER_MIN*, default 30)	Fallback without calling the API	Created
Low confidence	Kept and flagged	In the review queue

10.4 Resident assistant

The AI Assistant page (Figure 17) is a chat for residents. For each question the server builds the context itself from the signed-in user's flat: balance, due date and overdue days, unpaid bills, the latest payment, the flat's payment reference, the ten most recent complaints, recent notices and amenity timings. Nothing about the flat comes from the browser. The system prompt makes the

assistant read-only, forbids revealing other residents' data, and says not to invent amounts or payment details. The message is wrapped in tags as untrusted text, and history is limited to the last ten turns of 500 characters each. The route is limited to 30 requests a minute per user.

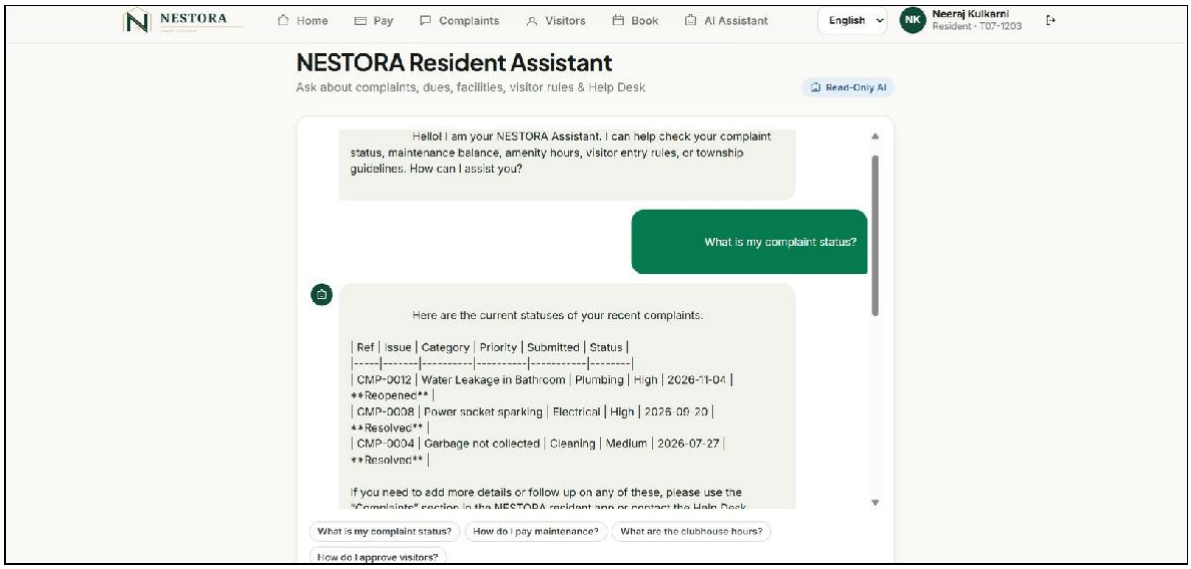


Figure 17: Resident Assistant page with the "Read-Only AI" badge, a complaint-status answer and suggested questions.

Read-only behaviour comes from the prompt and from the fact that the chat route has no write path, not from a separate filter on the user's message. If Groq is not configured or fails, the reply is a fixed apology. Tests (*chatbot.test.js*) check the prompt contents, isolation between two residents, 401 and 403 for other callers, that an injection attempt changes no record, and the fallback, all using an injected *fetch*. The answer shown in Figure 17 is formatted as a raw markdown table; the page does not render markdown.

10.5 What is rule-based, what is a model, and what is ordinary code

Kind	Parts
Model (Groq)	Classification text fields and confidence, duplicate similarity and explanation, assistant answers.
Rules	Emergency regular expressions, keyword triage, duplicate pre-scoring, PII redaction, output validation, review thresholds.
Fallback	Keyword classifier, rule-only duplicate score, fixed assistant reply.
Ordinary backend	Billing, matching, reconciliation, analytics, visitors, bookings. None of these use a model.

Verification status. The Groq transport and every failure path were exercised in tests with an injected *fetch*. That shows the code handles those responses. It is not evidence that *api.groq.com* was reached, and the one live test is skipped without a key. See section 12.4.

11. Security, Privacy and UX Decisions

11.1 Controls

Area	Control
Secrets	*GROQ_API_KEY*, *DB_PASSWORD*, *JWT_SECRET* are read from the backend environment. *.env* and *.env.** are git-ignored; *.env.example* holds placeholders. A test scans the repository and the browser files for secrets.
Authentication and authorisation	HS256 JWT with an eight-hour life, a role array on every route, residents confined to their own flat. The server will not start in production without *JWT_SECRET*.
Input	Length, enum, date and phone checks; malformed JSON gives 400; request bodies over 6 MB are refused; photos must be image data URIs under about 900 KB, three at most; SQL is parameterised.
Rate limits	Per-IP API (600 a minute) and OTP (20 a minute) limits, per-user limits for complaints (10), AI analysis (60) and chat (30), and a public-form limit. Limit hits return 429 with Retry-After. Counters are in memory.
Logging	Keys, passwords and complaint text are not logged. The database description printed at start-up has no password.
Headers	*X-Content-Type-Options: nosniff*, *X-Frame-Options: DENY*, *Referrer-Policy: same-origin*, *Cache-Control: no-store* on API responses.
Database transport	*DB_SSL* or *?ssl=true* turns TLS on with certificate verification; *DB_SSL_CA* supplies a private CA such as Aiven's. Verification can be switched off only through *DB_SSL_REJECT_UNAUTHORIZED=false*, which is meant for development.
Privacy (DPDP Act 2023)	Notice on the gate screen, minimal data sent to the AI provider, visitor photos deleted after 90 days, audit trail of staff actions.

11.2 Interface decisions

The staff views show the classification method, confidence and reasons; residents see none of it. A red safety banner explains an escalation was by rule. Data screens have loading, empty and error states and success toasts. A demonstration banner ("no SMS is sent and no real payment is taken") appears on every resident and staff screen in the captures. Layout checks at 390, 820 and 1440 px for the help desk were recorded in the earlier revision of this document; they were not repeated for this one.

12. Testing and Quality Assurance

Tests use Node's built-in runner (*npm test* runs *node --test test/*.test.js*). By default every test file builds its own in-memory SQLite database from the same *schema.sql* that MySQL uses. *test/helpers.js* can run the suite on a real MySQL server instead when *TEST_DATABASE_URL* and the matching credentials are set, but that wipes and re-seeds the tables, so it must only point at a throw-away database. It has not been run for this document. The default run is therefore a SQLite run, not a MySQL run.

12.1 Latest run

Run on 7 November 2026 with Node 22.23 on Linux, from a fresh clone of commit *804c9c8*, using the command *npm test*, without *GROQ_API_KEY* and without installing dependencies (the sandbox had no package registry access).

Total	Passed	Failed	Skipped
122	120	1	1

The skipped test is "LIVE Groq API: real classification returns a valid, validated result". It runs only when *GROQ_API_KEY* is set.

The failed test is "MySQL SSL configuration: disabled, enabled without CA (public CAs), and enabled with custom CA". It fails with *mysql2 is not installed* because *openMysql()* needs the package and this run had none. That is a property of the sandbox, not evidence of a code defect. It needs a normal *npm install* followed by another *npm test* to confirm. The same test would be expected to pass then, but it has not been observed passing here.

Earlier figures. The previous version of this document reported 99 tests (98 passed, 0 failed, 1 skipped). Running that earlier source tree again in this session gave the same 99 / 98 / 0 / 1. The count is higher now because *chatbot.test.js*, *duplicates.test.js* and the MySQL SSL test were added.

12.2 What the tests cover

Area	Covered (files)
Database connection and config	Round trip, engine name, migration record, config from environment, missing *DATABASE_URL*, SSL settings (*database.test.js*)
Billing and persistence	Repository round trips, sample figures reproduced from rows, idempotent bill generation (*database.test.js*, *billing.test.js*)
Payment matching	Outcomes on the sample CSV; UPI plus amount goes to review; overpayment to review; import never posts (*matching.test.js*)
Duplicate UTR and payment reference	Blocked in code and by unique keys; one bank transaction cannot post twice; concurrent posting yields one commit; rollback is atomic (*matching.test.js*, *billing.test.js*, *database.test.js*)

Area	Covered (files)
AI classification	Valid answer stored with method and model; request contains no PII and the key only in the header (*ai.test.js*)
Invalid AI output	Validator rejects each bad field; retry with feedback; two failures fall back; unknown category never stored (*ai.test.js*)
AI failure fallback	Network, timeout, 401, 429, 500, empty body, no key, call budget (*ai.test.js*)
Emergency escalation	Fire, gas, lift and shock stay Emergency when the model says Low, fails or returns junk; prompt injection; cannot be downgraded; Hindi (*ai.test.js*, *triage.test.js*)
Review and correction	Low-confidence flag, review queue, confirm or correct with an audit row (*ai.test.js*)
AI complaint preview	Analyze endpoint output, safety rules not overridable, review and correction of the suggestion (*ai.test.js*)
Duplicate complaints	Scoring, Groq comparison with failure handling, resident view hides other flats, link and unlink rules, circular and resolved targets refused, audit (*duplicates.test.js*)
Resident assistant	Prompt is read-only and carries only own data, history capped, resident isolation, 401/403, injection, billing answers from stored data, fallback (*chatbot.test.js*)
Roles and security	Route × role matrix, cross-flat isolation, token tamper and expiry, secrets scan (*security.test.js*, *api.test.js*)

Browser checks with Playwright against the SQLite dev runner were recorded in the earlier revision (resident sign-in, a gas-leak complaint, the help desk escalation banner, the classification card and the refusal to lower an emergency, plus overflow checks at phone, tablet and desktop widths). They were not repeated for this revision.

12.3 Local MySQL check

On 5 November 2026 the application was run against a local MySQL server. These results come from the earlier revision’s record and were not repeated on 7 November, because the sandbox has no MySQL server. No credentials or connection strings are recorded here.

Check	Result
MySQL Server	8.0.46
Database	*nestora_db*
npm run db:init	Completed
Schema	26 tables confirmed
users / *complaints* rows	9 and 11
npm start	Connected to MySQL
GET /api/health	*{"ok":true,"database":"mysql","time":"2026-11-05T08:31:23.744Z"}*

Check	Result
npm run db:seed	Inserted nothing, because the database already had data. Not counted as a successful seed.

This shows the adapter connected to a local server, the schema initialised with the expected number of tables and the health endpoint reported MySQL. It does not include the automated suite on MySQL, and seeding an empty MySQL database has not been verified. No reset was run, and none is recommended on a database that holds data. The number of tables was re-counted in the schema file on 7 November and is still 26.

12.4 Groq status

Implemented and optional. **Live Groq integration is not verified by this document.** The live test is skipped and no run here made a request to *api.groq.com* (outbound access from the sandbox was not available and no key was supplied). Two screenshots (Figures 8 and 10) display an AI badge with model *openai/gpt-oss-120b* and a confidence value. They suggest that a request succeeded in whichever environment was captured, but that environment is not recorded and the result cannot be reproduced from this document, so it is treated as an observation only. A key-holder can confirm the live path by running *npm test* with *GROQ_API_KEY* set.

12.5 Live site and production database

The reachability of the live address was checked on 7 November 2026 from the sandbox, and the sandbox's outbound proxy refused the connection to *nestora-new.vercel.app* (a 403 on the proxy tunnel). The result says nothing about the site itself. The GitHub repository could be cloned anonymously, which confirms it exists and is public. Nothing in this document confirms the live site's front end, back end, production database, sign-in or AI integration.

Question	Status
Website reachable	Not verified (blocked from the sandbox)
Front end loaded	Not verified. The screenshots are of a running copy of the app; the environment is not recorded.
Back end API working in production	Not verified
Production (Aiven) database connected	Not verified. The local MySQL check does not cover it.
Sign-in working in production	Not verified
AI working in production	Not verified

12.6 Executed and not executed

Item	Status
npm test (SQLite)	Executed on 7 November 2026: 122 tests, 120 passed, 1 failed (*mysql2* missing in the sandbox), 1 skipped (live Groq).
Same suite after *npm install*	Not executed
Browser checks	Recorded earlier; not repeated
Local MySQL init, schema check, start, health	Recorded 5 November 2026; not repeated
Local MySQL seed on an empty database	Not verified
Automated tests against MySQL	Not executed
Live Groq call	Not executed
Vercel deployment, Aiven database, production AI	Not verified

13. Assumptions, Limitations and Future Improvements

13.1 Assumptions

- MySQL 8.0.16 or newer, InnoDB, utf8mb4. The local check used 8.0.46.
- One township and one currency (INR). The committee decides the late-fee policy; none is applied until it is configured.
- The demo OTP and test-mode payments are deliberate for evaluation. No SMS is sent and no money moves.
- Groq's model list changes, so the default model is a setting to confirm against Groq's current catalogue.
- The stack is Node and vanilla JavaScript. An earlier requirements document named Spring Boot and React; the working system was kept instead of being rewritten.

13.2 Limitations and known issues

Limitation	Effect
Live Groq, Vercel back end and Aiven database not verified	Only code, tests with injected responses, and screenshots support these features.
One automated test fails in a clean sandbox	*mysql2* was not installed. Needs a re-run after *npm install*.
Tests run on SQLite	MySQL-specific behaviour, including concurrency, relies on the earlier manual local check.
Seeding an empty MySQL database not verified	The earlier seed run found existing data.
OTP attempts and rate-limit counters are in memory	They are not shared between instances. On Vercel, where requests can reach different function instances, *request-otp* and *verify-otp* may land on different ones and verification can answer "Request an OTP first". This follows from the code and has not been observed.
Duplicate suggestions on Vercel	The Vercel entry point does not pass the Groq client to the duplicate route (see 6.4), so only the rule score is used there. Classification and the assistant build their own client from the environment and are not affected.
Hourly photo purge runs on a timer	The timer is started when the server starts; whether it fires on a serverless host is not verified.
Demo OTP and test-mode payments	No real authentication or payment.
Photos are stored in the database	Fine for a demonstration, not for scale.
Analytics read rows into the application	Not tuned for 2,400 flats.

Limitation	Effect
Assistant has no rule-based answers and no action filter	Read-only comes from the prompt and from having no write path. When Groq is down it only apologises. Its answers are not rendered as markdown.
Keyword fallback is keyword-based	Its accuracy has not been measured.
No migration framework	Only the idempotent *schema.sql* is used.
Committed PDF in the repository	The repository contains an older *Technical_Design_Document.pdf* that still shows the old links. It was not modified here.
Screenshots	Captured with demonstration data. They are not evidence of the database, API or AI provider behind them.

13.3 Future improvements (not implemented)

Improvement	Purpose
Run *npm test* with *GROQ_API_KEY* set	Confirm the live Groq path
Check the Vercel deployment end to end (health, sign-in, a complaint, database rows)	Confirm production; the local results do not
Run the suite against a disposable MySQL database (*TEST_DATABASE_URL*)	Cover MySQL behaviour in automated tests
SMS gateway and a PCI-compliant payment gateway	Replace the demo OTP and test payments
Shared store (for example Redis) for OTP and rate-limit state	Needed before more than one instance
Object storage with signed URLs for photos	Replace photos in the database
SQL aggregation for analytics	Work at full scale
Use stored help-desk corrections to review thresholds and keywords	Improve classification
Render assistant answers as formatted text; add a server-side check for action requests	Readability and defence in depth
Versioned migrations	Manage later schema changes

13.4 Assignment evaluation checklist

Each item uses one of four labels: **Verified** (implemented and checked by a run or inspection recorded here), **Partially verified**, **Not verified**, **Not implemented**.

Quality (50%)

Item	Status	Note
Front end and back end integration	**Verified**	API tests call the real handler; screens in this document come from the running app. Production not checked.
Database persistence	**Partially verified**	Tested on SQLite; local MySQL check recorded earlier; automated MySQL run and production not done.
Authentication	**Partially verified**	JWT and OTP flow tested; demo OTP only; in-memory OTP state; production sign-in not checked.
Role-based access	**Verified**	Route × role matrix test passes.
Complaint management	**Verified**	Lifecycle, correction, review and link tests pass.
Visitor management	**Partially verified**	API flow covered; offline replay read in source, not browser-tested.
Payments (test mode)	**Verified**	Checkout, completion, partial payment and duplicate protection tested.
Real payment gateway	**Not implemented**	Test mode only, by design.
AI functionality	**Partially verified**	Tested with injected responses; live Groq not verified.
Responsive UI	**Partially verified**	One 360 × 800 capture here; wider checks recorded earlier, not repeated.
Loading, empty and error states	**Partially verified**	Present in the interface code; not exhaustively tested.
Automated testing	**Partially verified**	122 tests: 120 pass, 1 fails only for the missing *mysql2*, 1 skipped; SQLite only.
Security	**Partially verified**	Role, token, header, secret-scan and rate-limit tests; no external review; demo auth.

Problem solving (30%)

Item	Status	Note
Architecture	**Verified**	Layering inspected in source (section 2).
Database constraints	**Partially verified**	Present in the schema; exercised mainly on SQLite.

Item	Status	Note
Financial transaction safety	**Partially verified**	Transactions, unique keys and compare-and-set tested on SQLite; MySQL concurrency not tested.
Duplicate prevention: payments and UTRs	**Verified**	Tests pass.
Duplicate prevention: complaints	**Partially verified**	Rule scoring and link rules tested; Groq comparison only with injected responses.
AI validation	**Verified**	Validator and retry tests pass.
Emergency safety rules	**Verified**	Tests pass, including model Low, failure, junk and injection.
Failure fallback	**Verified**	Each failure kind is simulated; real-world outages not observed.
Human review	**Verified**	Review queue, confirm, correct and audit tested.

Communication (20%)

Item	Status	Note
Clear TDD	**Verified**	This document, checked page by page after export.
Architecture diagrams	**Verified**	Figures 1, 2 and 16.
Database documentation	**Verified**	Section 3, from *schema.sql*.
AI explanation	**Verified**	Section 10, from *server/ai/*.
Test evidence	**Partially verified**	Latest run recorded; MySQL and live Groq runs absent.
Deployment documentation	**Partially verified**	*deployment.md* commands checked against *package.json* and source; the Vercel steps were not executed.

Items outside the three groups

Item	Status	Note
Production Aiven / Vercel database	**Not verified**	See 12.5.
Live Groq API	**Not verified**	See 12.4.
Live site reachable, sign-in and AI in production	**Not verified**	See 12.5.
Real SMS OTP	**Not implemented**	Demo OTP only.

14. Mobile Responsive Design

Figure 18 is the resident home in a browser device-emulation view set to Samsung Galaxy A55 at 360 × 800, with the emulation toolbar visible at the top. It is an emulated view, not a photograph of a phone, and what follows is limited to what the capture shows.

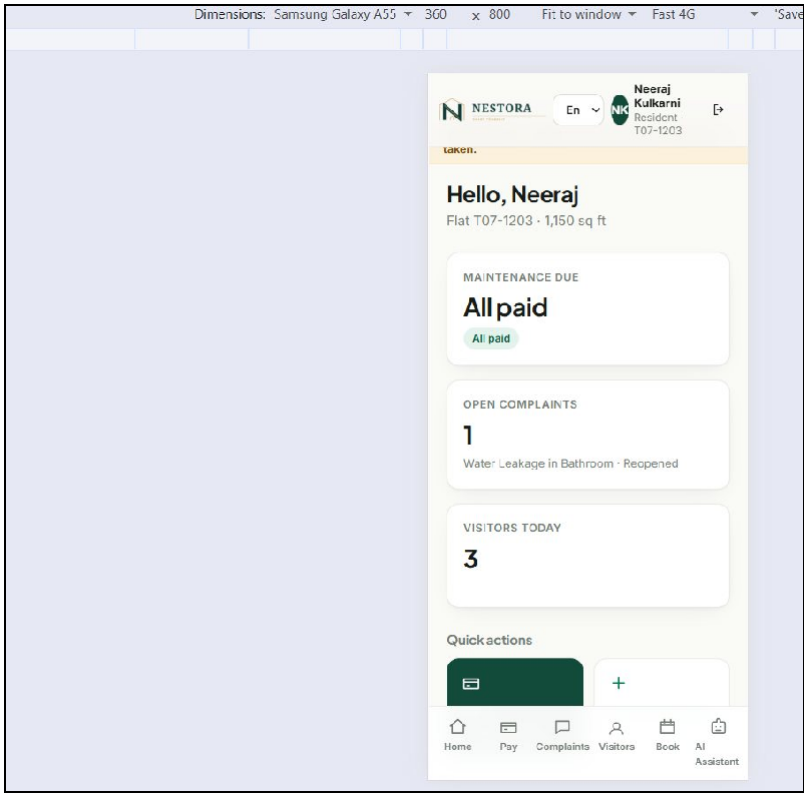


Figure 18: Resident home in a Samsung Galaxy A55 emulation (360 × 800).

Element	What the capture shows at 360 px
Top bar	Desktop navigation links are gone. The bar holds the logo, a short language selector (“En”), the avatar with name, role and flat wrapped over three lines, and a sign-out icon.
Demo banner	Only the end of the banner text (“taken.”) is visible under the top bar, so it appears clipped here.
Cards	Maintenance due (“All paid”), Open complaints (1, “Water Leakage in Bathroom · Reopened”) and Visitors today (3) are stacked full width. The flat and area (T07-1203 · 1,150 sq ft) are in the greeting.
Quick actions	A heading and a two-column tile grid, partly below the fold.
Bottom navigation	A fixed bar with six icon-and-label items: Home, Pay, Complaints, Visitors, Book and AI Assistant (the last label wraps to two lines).

Scope. One viewport was captured for this revision. The 390, 820 and 1440 px checks mentioned in section 11 come from the earlier record. No claim is made about every device. An earlier capture of the same view for flat T12-0408 showed five bottom items, before the AI Assistant tab was added; it is not repeated.